

2i SD WYKŁAD XV

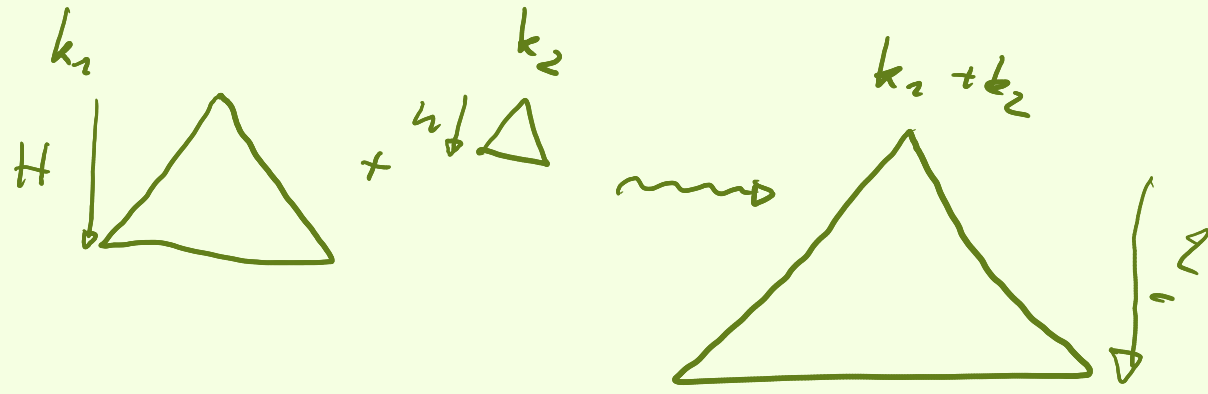
Analiza
 amortyzowana

$$\log n + \frac{1}{n} \leq \log(n+1)$$

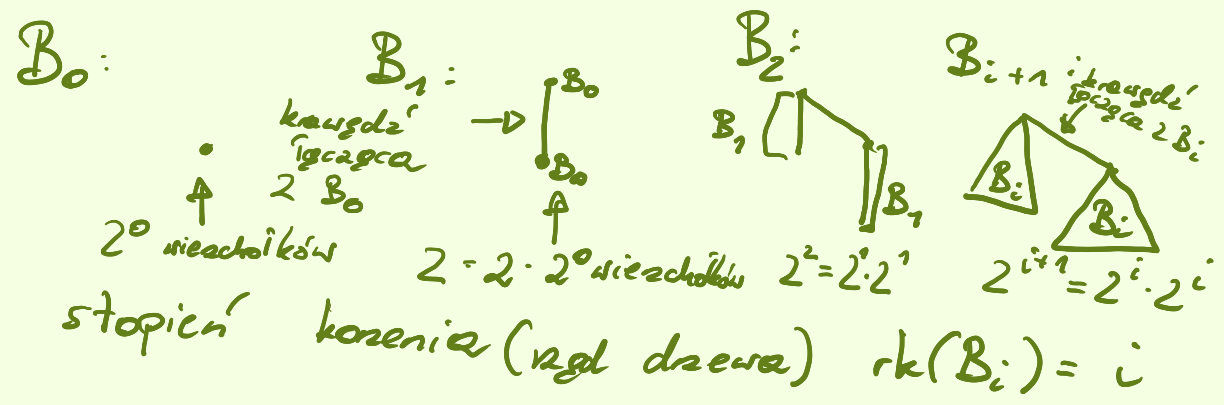
$$\frac{1}{n} \leq \log\left(\frac{n+1}{n}\right)$$

$$[0, 1] \quad \log[2, e]$$

Kopce dwumianowe: (kolejki priorytetowe złączalne)



Drzewa dwumianowe:



FAKT:

B_i zawiera 2^i liściów (bo B_{i+1} zawiera 2 razy więcej, niż B_i).

FAKT:

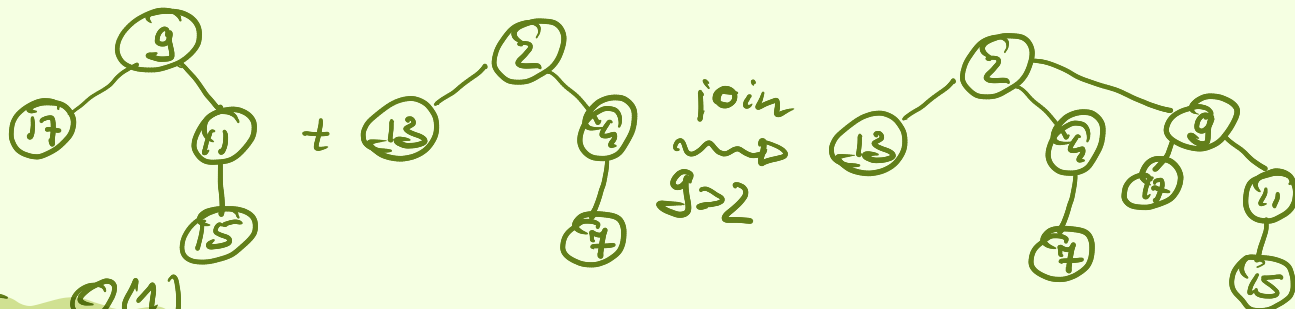
Na k -tym poziomie mamy $\binom{n}{k}$ liściów.

Kopiec dwumianowy - kolekcja drzew B_i - w każdym węzłach pomieszczony jest 1 klucz w porządku kopcowym.

Operacja join/link: Łączymy 2 drzewa równych rozmiarów:



Przykład:



Koszt $O(1)$

Przyjmujemy, że:

Każdy węzeł pamięta wskaźnik na listę (cykliczną) swoich dzieci.

Operacje:

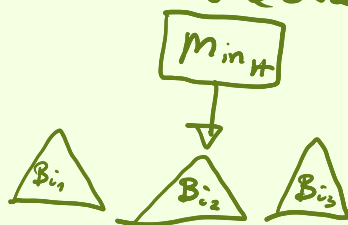
- (a) insert $\rightarrow \text{meld}(H, \text{makeheap}(k))$
- (b) min
- (c) deletemin
- (d) makeheap(k) - utworzenie nowego kopca z kluczem k
- (e) meld(h_0, h_1) - z dwóch kopców zrobić 1.

Kopiec H posiada komórkę Min_H = wskaźnik na korzeń zawierający najmniejszy klucz

ad. a - jak zwykle

ad. b - oczywiście (patrzmy na Min_H)

ad. c - jak zaktualizować Min_H ??



ad. e:

musimy je wykonać gorliwie (eager) / leniwie (lazy)

W wersji eager

k_i kopiec zawiera ≤ 1 drzewo B_i



$$H[i] = \begin{cases} \text{wskaźnik na } B_i & \text{jeśli istnieje} \\ \text{null} & \text{jeśli } B_i \notin H \end{cases}$$

Przykład:

Niech $n = 52$ (liczba kluczy w kopcu):



32 węzła i + 16 węzła kół

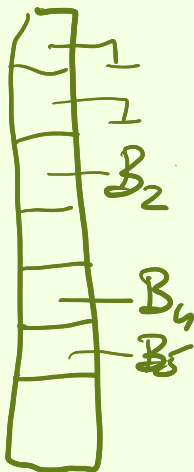


H''

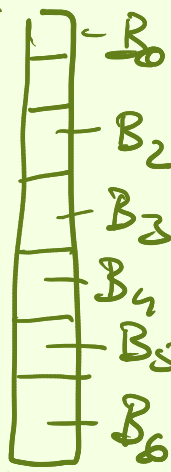


+ 4 węzła kół = n węzła kół

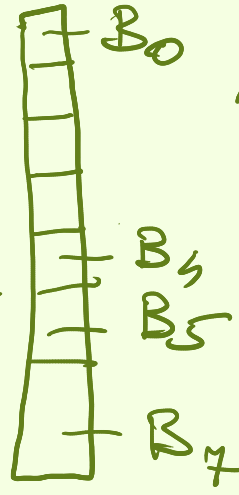
H' :



H'' :



H'''



B_2^H, B_0^H

Przechowujemy kopiec zawierający dla $i=0$ do n wykonujemy:

jeżeli $B_i \in H' \wedge B_i \notin H''$

lub vice versa:

jeżeli $B_c \neq \emptyset$:

$B_c \leftarrow B_{i+1}$

wpp:

$H''' \leftarrow H''' \cup B_{i+1}$

jeżeli $B_i \in H' \wedge B_i \in H''$:

$H''' \leftarrow H''' \cup B_{i+1}$

$B_c \leftarrow B_{i+1}$

Niech B_i^H oznacza:

$$B_i^H = \begin{cases} 1 & B_i \in H \\ 0 & B_i \notin H \end{cases}$$

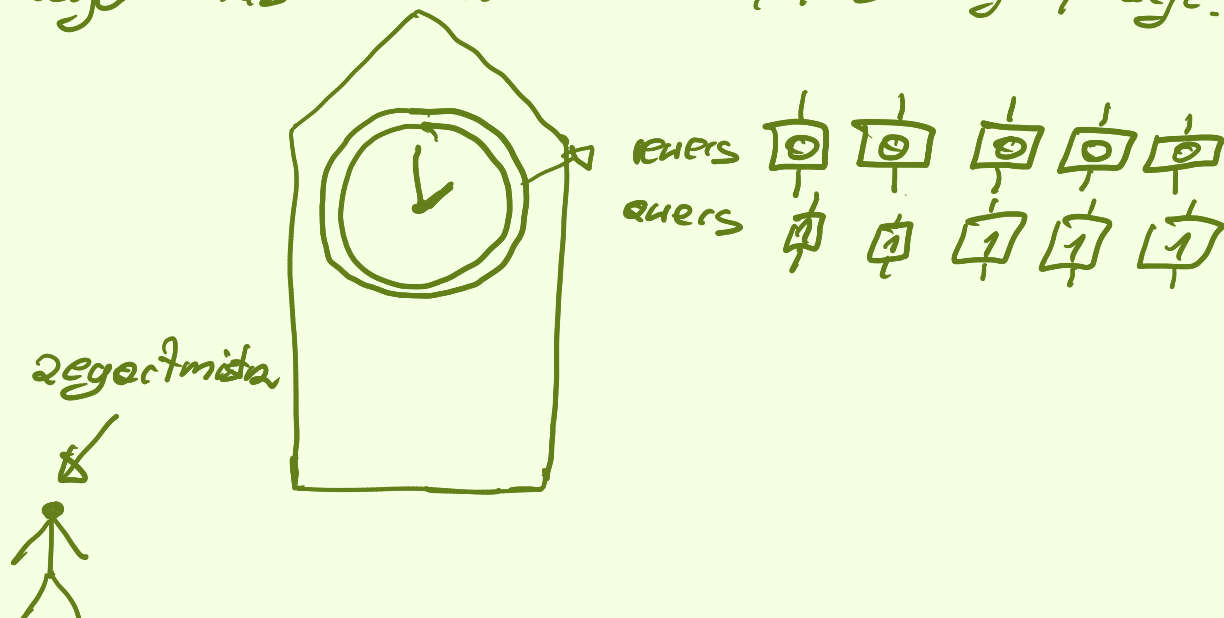
Rozet meld $O(\log n)$

H	B_2^H	B_4^H	B_5^H	B_0^H	B_3^H	B_1^H	B_6^H	B_7^H
H'	0	1	1	0	1	0	0	0
H''	1	1	1	1	1	0	1	1
$H' \oplus H''$	1	0	1	1	0	0	1	1

Koszt amortyzowany:

Ciąg operacji α : $op_1, op_2, op_3, \dots, op_n$:

Interesuje nas średni koszt pojedynczej operacji:

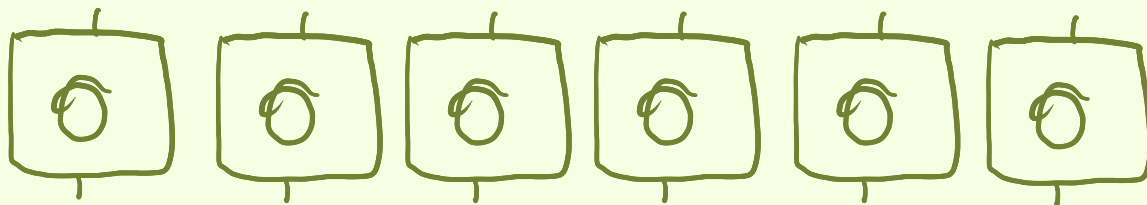


Zegarmistrz ma inkrementować wartość zegara.

Jednakże tabliczki są ciężkie i niestety kosztuje go dużo energii. Związek z tym ma to funduje mu ciasteczka i jedno wystarczy na obrócenie 1 tabliczki.

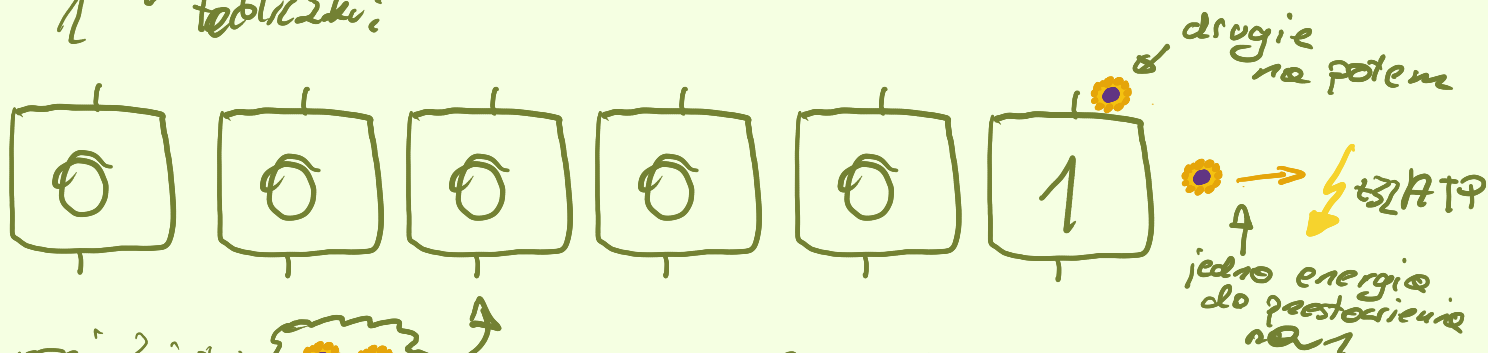
Zegarmistrz przychodzi z 2 ciasteczkami i wybiera jedno obraca tabliczkę, jeżeli przestawia 0 na 1 to zjada 1 i zostawia drugie za tą tabliczką na potem. To, które zostawia na potem zjada tylko wtedy, gdy obraca tę tabliczkę pod którą było to zapasowe ciasteczko, w dodatku jest to obrót z 1 na 0 (bo jak wcześniej było 1 to obrócić jeszcze wcześniej 0 na 1 i odłożyć ciasteczko).

Jak mu się skończy (lub zostanie tylko te odłożone na potem pod niedobrzoną tabliczką) idzie po 2 następne.

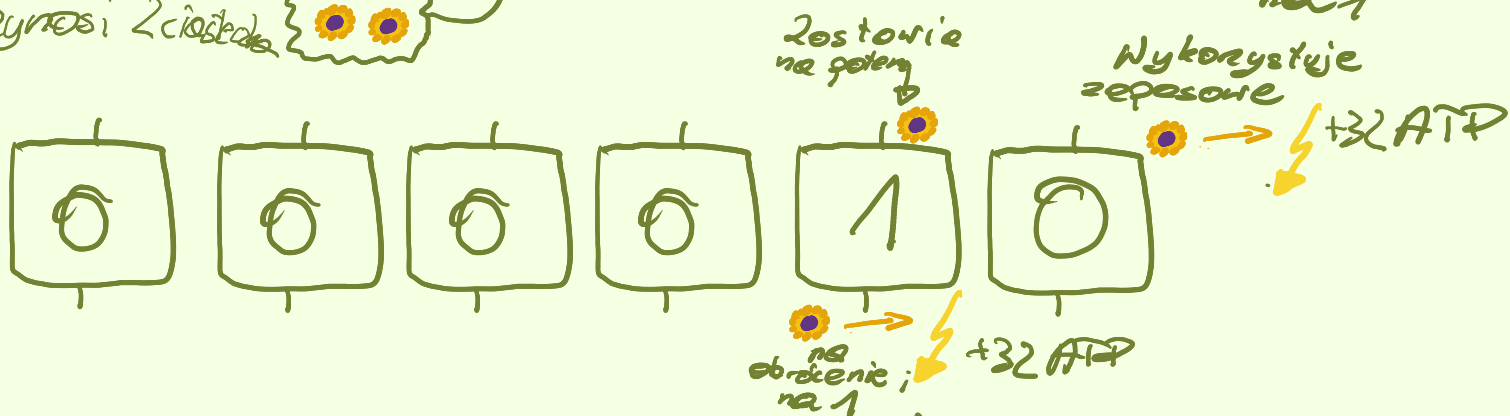


Początkowy stan

Zegarmistrz przychodzi z 1 poje ciasteczki i jednę zjada w celu nabrania siły do obrotów 1 tabliczki:



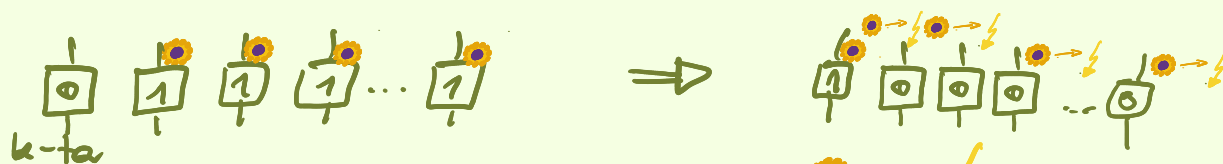
Przynosi 2 ciasteczki



Kolejne 2 ciasteczki



Perce zauważamy, że zegarmistrz musi obrócić k-tą tabliczkę z 0 na 1 to wcześniej obrócił z 1 na 0 (przenosił carry) i zrobić to bez wracania po ciasteczki, bo miał ciasteczka odłożone.



Kolejne 2 ciasteczki



obrotów k-tej na 1

No ale w wyniku inkrementacji tylko o indeksach większych niż k nie ulegną zmianie, czyli zegarmistrzowi udało się dokonać inkrementacji całej wartości zegara,

mimo, że zainicjował tylko 2 ciasteczko.

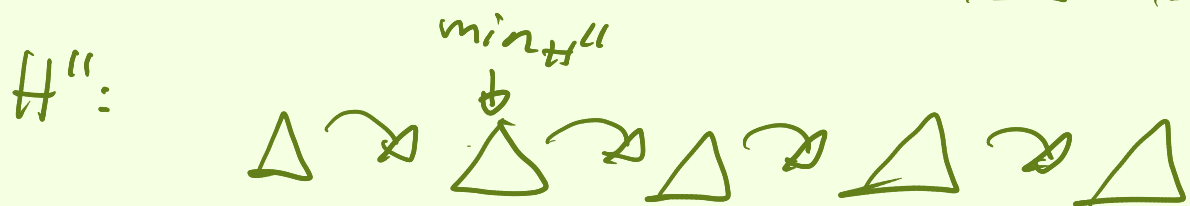
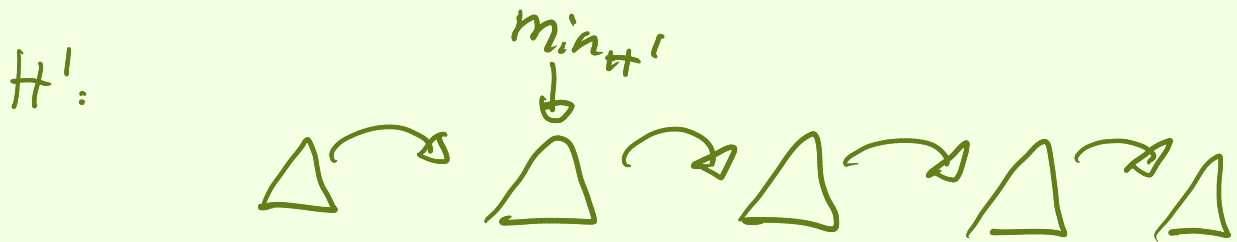
Czyli do n inkrementacji potrzebujemy $2n^{(1)}$ ciasteczek (bo wystarczy, że przyniesiemy po 2 na każdą inkrementację), mimo że pesymistycznie potrzebujemy k ciasteczek na inkrementację k -bitowej liczby (które ma same 1), co daje nk w przypadku n inkrementacji.

Koszt (1) jest zamydlony, a średni koszt inkrementacji to $\frac{2n}{n} = 2[\text{ciasteczko}]$.

Meld wykonane w sposób łatwy:

Perła kopiec może zaskładać wiele kopci Z_i :

Drewno kopca łączymy w listę (cykliczną)



$\text{meld}(H', H'')$  koszt $O(1)$

Delete min:

- usuwamy korzeń drzewa zawierającego minimum
- gałęzie drzewa włączamy do listy drzew kopca.
- redukujemy liczbę drzew (tak, by H_i a kopco było $\leq 3_i$)

Każdej operacji przydzielamy pewną liczbę jednostek kredytowych (ma starczyć na stałą liczbę operacji maszyny VRAM, podobnie jak ciasteczko sterza na przekroczenie tabliczki w zegarze).

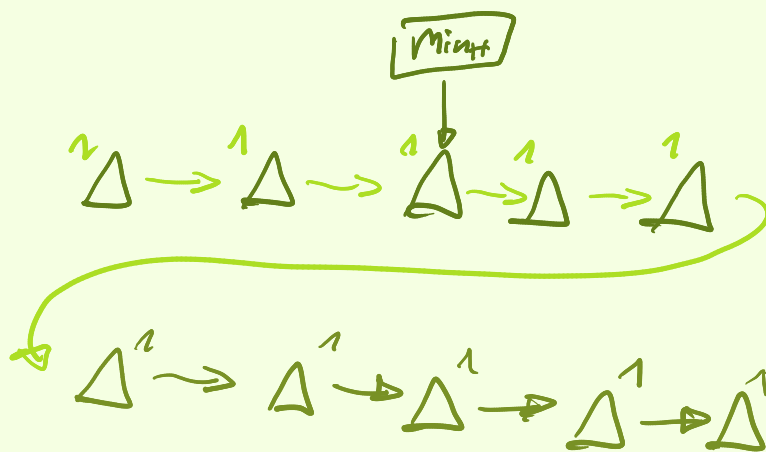
Niezmiennik kredytowy

Każde drzewo kopca ma 1 jednostkę [jk] kredytowe na swoim koncie.

Podział jednostek:

- ~ insert - 2 jk (ustawienie i jedno w zeposie)
- ~ meld - 1 jk (tylko złączenie)
- ~ min - 1 jk (tylko odczyt)
- ~ mergeheap - 2 jk (petla, insert)
- ~ deletemin - $2 \log n$ jk.

na jasnozielono - jednostki kredytowe.

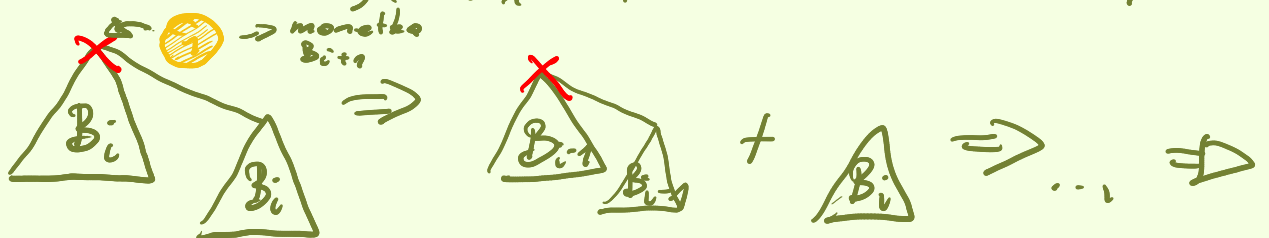


Przed deletemin:



Teraz inwestujemy w deletemin $2 \log n$ monetek (jk).

usuwamy minimum (monetkę drzewa B_i zużyjemy na to minimum). B_{i+1} rozpruje się na i drzewa jak niżej.



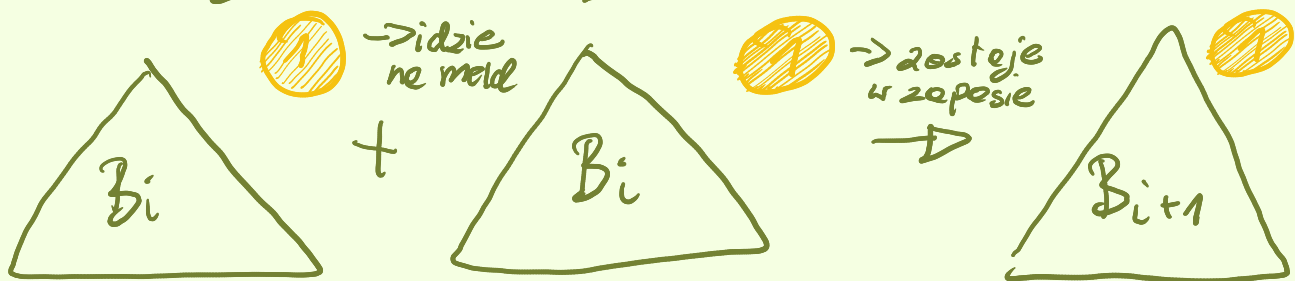
Powstało now i drzew bez monetki.

Skoro n - liczba kluczy w kopcu, a B_{i+1} zawiera 2^{i+1} kluczy, to $i+1 \leq \log n$ (bo $2^{i+1} \leq n$).

Teraz przypisujemy powstałym drzewom 00 2 monetki i dołączamy do kopca. 1 się zużyje na insert. Zatem nowe kopce mają po jednej monetce po insercie. Z ten sposób zużycie $\leq 2 \log n$ monetk



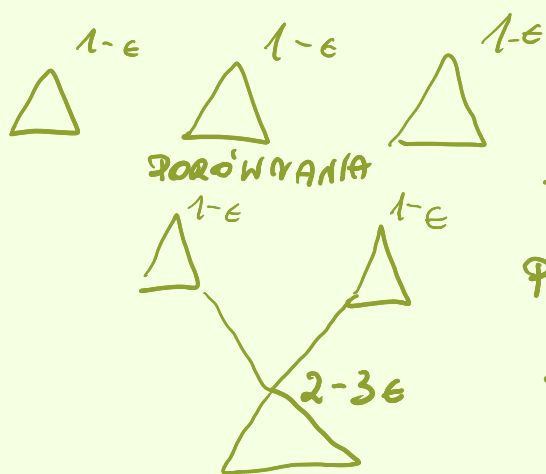
Teraz pozostaje złączyć kopce. Korzystamy z niezmiennika kredytowego. Wszystkie, które dodaliśmy wcześniej miały 1 monetkę, czyli robiąc melda mamy do dyspozycji 2 ze złączonych drzew:



Ponieważ co niezmiennik dalej jest zachowany

UWAGA do konsultacji u Łukasza

Trzeba jeszcze zaktualizować minimum - w tym celu należy powtórzyć się po drzewach i za to też zapłacimy. O monetkach myślimy w ten sposób, że każda starza na stałą liczbę operacji, czyli np. za 1 monetkę możemy opłacić porównanie minimum 2 drzew (czas stały) oraz za meldo (też stały).



→ tu płacimy ϵ (powiedzmy...

$\epsilon \leq \frac{1}{3}$ monetki)

Przychód na korekcie	Koszt meldo	Zostaje
$1-\epsilon + 1-\epsilon = 2-2\epsilon$ z jednego drzewa	ϵ	$2-3\epsilon \geq 1$

Jeszcze 1 rzecz - znajdowanie drzew o takiej samej rozmiarze też może być robione w czasie stałym.

Uwaga do konsultacjiach UMBA

~ do aktualizacji minimum, jeżeli $\exists$ kopce które **NIE** brały udziału w złączeniu dalej mają $1-\epsilon$ monetek - i to jest problem, bo zaburza niezmiennik kredytowy..., właśnie po to jest drugi $\log n$, bo po usunięciu minimum mamy $n-1$ elementów w drzewach

$B_0, \dots, B_k$ ($k \leq \log(n-1) \leq \log(n)$). Co najwyżej

1 drzewo każdego rodzaju nie brały udziału w złączeniu (z tobie B_i dołączy B_{i+1} , a przecież założyliśmy, że ich nie złączeliśmy). Czyli będzie ich co najwyżej $\log(n)$.

TODO - przepisać całą analizę na nowo